

Suggestions for the Belmont University Project

- **Improvements in point storage implementation**
 - Avoid boxing when reading text. In `TextPointStore.java` `List<Integer>` is used and then copied to arrays, which creates many objects and doubles the work. It is better to read the first line, allocate `int[] x` and `int[] y` with the exact size, and fill them directly.
 - Export points to avoid reloading/copying them per thread. `TrianglesUtils.java` recopies all points into `xCoords/yCoords` when starting each partial count. In thread/process mode this can repeat several times.
- **Avoid trashing when there are many threads.**
 - Up to 256 threads are allowed in `ThreadTriangles.java:69`, but it is not limited by logical cores. Limit threads by CPU, not by points. Use a maximum equal to the logical cores (or logical cores minus 1). Keep 256 as a hard safety cap, but apply a lower operational cap by default.
 - Each thread recopies all points in `TrianglesUtils.java`. Load all points once at startup, store them in a shared immutable structure, and have each thread only compute its range using that same structure.